Corncob Redux - Mission Builder

Lost Engines - Corncob Redux
Reviewed edition of the original game documentation.
Controls and changed rules describe Redux defaults.
Original credits, source tables and archival details are retained.

Contents - select a chapter to jump to it:

 1. Instructions on how to use Corncob's mission builder.
 2. How to use the mission builder
 3. When you turned on the cursor, note that the numbers came back onto
 4. Objects can be rotated in place by holding down the 'a' key, and then
 5. Note that this applies both the up down, and left right tile centers.
 6. Mission builder: why is it doing that?
 7. ; The following is taken from the sources of Corncob. This is the
 8. ; The following is a table of various properties of objects. You cannot
 9. dw 0   ;34
10. ; Special bit definitions:, hit F9 to set an object's special bit

Instructions on how to use Corncob's mission builder.

Copyright 1992 Pie in the Sky Software

Theater files and tower files
Redux integrates the mission builder into the game. Choose Select campaign / world group, then Mission builder. Choose a world to edit; the editor uses your current pilot's saved copy. F5 saves and returns to the menu. A copy of the edited definition and its tower text is exported into custom_worlds inside your Redux save directory. There is no need to run 3.EXE, copy 3UNIV.DAT, or use DOS batch files.

The following description of DEF and TWR structures and the source tables is retained for reference. Internal object names and assembly tables are reproduced as written; they are not commands to type into the native executable.

For every .def file in the DEF subdirectory, there is also a corresponding
.twr file. The .twr file contains intelligence information that the user
will see when he requests it via F2, or at the beginning of the mission. The tower files are just normal text files with 9 formfeeds. (Control L) The body of text between each formfeed is a block of text that the user will see when he starts from that particular airstrip. The easiest way to create your own tower file is to modify an existing tower file, rewriting only the actual mission info, and leaving the airstrip names and formfeeds alone.
Some text editors may not like the formfeeds. The tower files were originally created with MKS vi editor, which handled the formfeeds with no problem.

The start positions for the missions are programmed into the tower files.
Each mission description has a line with the characters Sxyth in it. The numbers following these 5 characters contain the x, y, z, and yaw angle.

To program in your own start coordinates, simply use the Map mode to position the viewpoint where you want the pilot to begin the mission. Hold down the left shift key, and use the keypad arrow keys to change the angles of the view point. When you are satisfied with the position and angle, simply copy the x,y,z and yaw angle numbers off the bottom of the screen.
Then edit the tower file, and insert these numbers after the Sxyth characters. Make sure you leave spaces between the numbers, and don't put any extra spaces at the beginning of the line.
Note that you can only set the yaw angle (compass heading type) of the plane's start. You can't change the initial pitch or roll. IE you can't make the plane start out flying anything but level at the start of a mission. You can only change the compass heading which the user will start out at.

The sun position, air pressure and gravity.

Tower files can start out with a line like the following:

X 6000 300 12 -48 10000 0 1500

Instructions on how to use Corncob's mission builder. (continued)

The X marks that the numbers on this line specify the sun size, position and color. Also is the gravity, air pressure, star flag, and atmosphere extent.

X identifier for theater parameter line.

the 6000 is the angle of the sun in the sky. Range +-32767

300 is the size of the sun Range 10-1000

Color 12 is the color of the sun Color 0-15

-48 is the gravitational attraction Range 0 to -120

10000 is the air density at sea level 6000 is normal atmosphere 10000 max

0 is the star flag. 0 means no stars 1 means stars in sky

1500 is the atmosphere height. 6000 is Earth normal.

How to use the mission builder

To start a fresh universe copy default.def to 3univ.def. Default.def is a universe with just airfields in it.

Put the universe file in a file called 3univ.dat in the current directory.

Start the game with 3 -rs

When sitting on runway in plane, type m to go into map mode. Make sure the joystick is not drifting, use trim if needed.

Zoom in on the area to be edited with Joystick/Arrow keys, and pg up and pg down to change height. When close to surface, hit 'b' to enter the mission builder mode. When in MB mode, the gunsight disappears. IE, you can tell mission builder apart from map mode by checking for the existence of the gunsight on the screen.
Close to the surface is a height of about 30000-60000 cm. The numbers on the bottom of the screen read X,Y, and Z in cm. (Z is height)

When in MB mode, the Joystick/Arrow keys will change the viewing direction, just as when you are an ejected pilot. Just like an ejected pilot, the numpad keys '5', and 'ins' will move you forward or backward in the direction which you are facing. The constraint that you remain on the ground is however lifted.
When slewing around in this method, the non-keypad number keys will change your walking rate. IE '1' makes you move slowly, '8' makes you move quickly.
Caps lock will instantly level you out, so you are looking at the horizon.
This is usefully when you want to move in a perfectly level direction.

Now to select an object for moving/deletion/changing, you must bring up the 3d cursor. Do this by tapping the left shift key. A rotating ball of spheres should appear in front of you. If you move about, the cursor remains in its position of creation. Do not move too far away from the cursor, or the program will become confused because it cannot operate on a cursor which is in far object memory. Be careful when using pgdn especially. If you are going to move a far distance, turn off the cursor with the left control key first.

If you wish to move far away, tap the control key to make the cursor disappear. Also, you may return to the map mode with the cursor either lit

How to use the mission builder (continued)

or unlit, and move around in map mode, and then back to builder mode via the
'b' key.

  The cursor can be moved with the Joystick/numpad arrow keys, if the alt
key is held down while doing so. The pgup and Pgdn will move the cursor
up and down, however your viewpoint will also change height at the same time.
(IE, when you move the cursor up or down by holding down the left alt, and
pgdn or pgup, the cursor will remain at a fixed position relative to you, but
you AND the cursor will move up or down.)

When you turned on the cursor, note that the numbers came back onto

the bottom of the screen. These are no longer the viewing coordinates, these are the coordinates of the cursor. The number 21 is the object type. When the cursor is displayed, the number is always 21. The last number in the list is the object quality. This can be the wickedness of an AAA, or in the case of a Headquarters orb, this defines the mission objective. As with many types of objects, the quality of the cursor is ignored.

  So anyway, when you are near the ground, and the cursor is on, tap the 'c' key to create an object. The default object is a headquarters orb. It should create an orb at the cursor position which is big and blinking.
  Each type of object has an object number. Object number 0 is a generator
orb. Object number 1 is a mind bender, object number 2 is a plane. A list
of objects and their type numbers is included with this doc.

  Use the non-keypad += keys to cycle through the available objects. The
current object should be sitting there blinking slowly. Each time you press the non-keypad + key, the object will metamorph to the next object type. IE the HQ orb will turn into a mindbender. As you cycle through all the different objects, they will appear at their default height above
the ground. This can be a pain sometimes, especially when creating missions
high in the air.

  As long as the object is blinking, you may move it around with the arrow
keys just like the cursor. To make the keypad arrow keys operate on the object instead of moving yourself, just hold down the left ALT key while you press the keypad arrow keys. The number keys will affect the motion rate
just like when moving about. Be careful not to lose track of where the object is. If the object doesn't appear to moving in the direction you are requesting, try tapping the non-keypad '1' key, to set the speed to slow.
  Note that objects will always move in a horizontal plane when moving them
with the arrow keys. An object's height can only be changed with the pgup and pgdn keys.

  A blinking object will be deleted if the 'd' key is pressed. IE the current object which you have just created (or selected) can be removed from the universe with the 'd' key.

  To make your blinking object a permanent object, press the left shift key. The currently selected object will stop blinking, and is now a permanent object just like any other object in the universe.

  If you wish to make changes to existing objects, then you must first make that object become the current object. (IE make the thing blink)
  To select your object, move the cursor close to the center of the object, then hit the space bar. The space selects the object which is closest to the cursor. BEWARE: Do not use the space bar if there are no objects even close to the cursor, you may get kicked out of the

When you turned on the cursor, note that the numbers came back onto (continued)

program with a critical error.
  Make sure the correct object is blinking, if it is not, then probably some other object (which perhaps you can't see from where your looking) is selected, and all changes you make will be happening to the poor hapless
object which you have unwittingly selected.

Objects can be rotated in place by holding down the 'a' key, and then

using the arrowkeys/Joystick to rotate the object. To lock the object to the nearest 90 degree plane, press the appropriate key.
NOTE: All objects can be rotated, but non-circular objects such as forcefields should NOT be rotated because the rectangle of collision of the forcefield will not be rotated with it's image.

  Some of the objects have a 'special bit' which can be set with the mission builder. For example, if a generator orb has its special bit set, then it will be a gravity orb. If a shopping cart has its special bit set, then it can only be destroyed by a human weapon, and will never die via collision with another object.
  This special bit is set with the F9 key. When an object's special bit is set, the object type number on the screen bottom will turn purple.

  Like the special bit, the F10 key will set an object's complete bit. The complete bit matters in only a few objects. This is the bit which keeps track of whether or not a mission objective has been accomplished. If you set the complete bit of an object, then the object type number at the bottom of the screen will flash.
  One use of the complete bit is to make portals which would ordinarily count as a mission objective, not count. IE say you wanted to make a mission where you had to fly by a bunch of portals on the way to your real
mission objective. If you just create the portals, then all of them would
count as mission objectives. IF you set their complete bits ahead of time,
then they will count has having been already flown through.

Zlocking
  Often one wants to make many objects all be created at the same height.
This can be accomplished with zlocking. Simply put the first of these objects at the desired height. Then while that object is still selected,
press the left alt and the 'h' key. This records the present z coordinate
into zlock-height register. Now create your next object. To force it to be the same height as the zlock-height, simply press 'h'. The z coordinate number at the bottom of the screen should turn black, and the
object should be the same height as the last one.
  The z coordinate number should turn black whenever the present selected
object's z coordinate matches those of the zlock-height register.

Notes on where to build
  Each theater is made of an 8x8 grid of tiles. These square tiles each can have a maximum of 100 or so objects. This limitation should never be approached, as this will create a critical error, thus propelling you swiftly out of the program will all your changes lost.
  The tile boundaries are mapped out with survey markers which can be seen on the ground, and also show up in map mode as a grid of yellow or black dots.
  During game play, only the objects in the nearest 4 tiles will be displayed. Therefore you can see that when the user is flying in the middle of a tile is always when one tile is swapped in, and another

Objects can be rotated in place by holding down the 'a' key, and then (continued)

out. In other words, the computer will have to do some extra work to swap out a tile, and read in a new one when you cross the middle of a tile. For this reason, it is a good idea not to put missions in the centers of tiles. If you do, then the game play will be continually interrupted for tile swapping during the acrobatics.

Note that this applies both the up down, and left right tile centers.

Picture a square tile with a '+' sign on it, where the plus sign is big enough to cover the whole tile. The imaginary lines of this plus sign show the areas where tile swapping will occur. Therefore, your missions should always be closer to the corners of the tile, then the tile centers.

  Notice that the frame rate will suffer dramatically if you try to put too much stuff close together on neighboring tiles. For example, you might want to make a mega-mission at the intersection of 4 tiles.

  You could put up to 100 objects in each tile, and still have it all fit together as one mission. However, in reality, so many objects to be calculated and drawn would lead to an absolutely terrible frame rate, and an unplayable mission.

  You must also be aware that even if you don't have 100 objects in a tile, you could still end up with a critical error during game play

  The reason is that passing KLA's, AA shells, and shopping carts all count as objects in that tile, even if they were created in one tile, and migrated to the other.

  If you look at the default theaters that came with the game, you will notice that most all of the bigger missions are made to be at corners of tiles, and well away from the airstrips. This is simply because of the need to keep the frame rate up. This is especially important for those players with machines slower than 25Mhz.

  You will notice that the frame rate is not so great at Sweetly Bending in Flatland because the missions were too close to the airstrip. This makes landing there difficult.

Notes on what to build

  Usually start with a generator orb. Set the quality to corresponding to what you want the mission object to be.

  Keep in mind that the many players will be bored with just a bunch of KLA's and deathballs. What makes a mission fun is not just a challenge, but a challenge that takes strategy to lead to victory. Examples: A mission where there are shells beneath an orb, so that if the play tries to take down the forcefields first by destroying the orb, then the shards that fly off the orb will trigger the shells, thus bring a whole slew of enemies upon the unwary player.

  How about putting shells down on the ground where they wouldn't normally
be in danger of triggering, but then but some high quality vehicles on the
outskirts, where they will follow the plane in, and set off the shells even if the flyer doesn't shoot the shells himself.

  The force fields open up a whole lot of possible structures that the pilot must navigate to successfully complete the mission. Beware of overuse of the solid or solid-shimmering fields, as they soak up the cpu time to draw. Use the wire frame versions where possible.

Mission builder: why is it doing that?

> Why does it seem impossible to get the height of objects right when
>they are pretty high up?
   The mission builder makes the pgup and pgdn keys have an effect
proportional to the height of the object. When the object is relatively
high up, the vertical steps are huge. This can be gotten around via
the 's' key. When this key is depressed, the pgup and pgdn keys will
always move a relatively small distance (1000 cm) regardless of the
object's height.

> Why are the objects always the wrong color, and sometimes the wrong
>shape when I go into the mission builder or map?
  The way the program is written makes the qualities of objects never
matter when in the mission builder. This is because the code which
makes the object the correct color etc is the same code which makes the
object move, shoot etc. Therefore if that part of the code were turned
on during the mission builder, then the objects would begin to move
and fire as soon as they were created. So unfortunately, you will have
to trust the quality number on the bottom of the screen to see the
actual
quality of the object.

> I worked for two hours on a mission, and when I tried to get out of
>the mission builder mode when finished, I got critical error 70, and
>lost all the changes.
   The mission builder is not perfect, and I typically only make small
changes, and then exit and back up the 3univ.dat file. This process
is relatively quick, and all the default theaters which came with the
game were created with the same mission builder which is available to
you. Occasionally you will lose stuff, but if you never stay too long
in the MB, and don't keep skipping back and forth between map and MB
mode, you should never lose too much.

>What are the things to avoid doing in the mission builder?
  At any one during game play or during mission creation, the computer
can only deal with the objects which are relatively close. When an
object
gets too far away to the observer, it is placed into the 'Far object
table'.
  Objects in the 'Far object table' are inactive, and are stored in
compressed form.
  The mission builder can operate on near objects only. Far away
objects
cannot be processed by the builder.
  It is VERY important that you never ask the builder to act on an
object
which is out of the near range. This will cause unpredictable behavior,
and critical error codes.
  Some examples of causes of this:

  Say you create an object on the ground, and then zoom your viewpoint
up
to 40,000 feet. Then you start moving the object around on the ground.
If
you move that object too far away, you will trap your newly created
object
in limbo, and you will be risking a critical error.
  Try to stay near your current (blinking) object at all times.

Mission builder: why is it doing that? (continued)

Say you start to build a mission, and then decide to scrap it. Deleting the last object will get you a critical error, since there are no objects at all to process.

Say you are using the keypad + and - keys to cycle through the near objects. You get impatient, and hit the - key repeatedly very fast. If you cycle through all the near objects, then the poor mission build tries to pick up a far object and trouble ensues.

All these limitations may make the mission builder seem like it isn't very useful, however with the proper habits, you can use the MB continuously for days without a mishap. This very program was used to create all 150 of the available missions in Corncob.

Just remember to only make one mission at a time, and make backups of the theater to another filename after every mission. Don't flip back and forth between map and mission builder mode. Start the program, make your changes, and exit back to DOS between each change. Take care not to move too far away from the current blinking object, and never travel any great distance withing the mission builder. If you want to move somewhere else in the theater, exit the program completely, and start it up again.

To exit the mission builder, you can hit escape or 'm' to return to map mode, then escape or space bar to return to flying mode.

Do not fly after returning from the mission builder, just exit, then copy 3univ.dat to carl.dat or whatever. Its a good idea to make backups in case of unexpected critical errors.

; The following is taken from the sources of Corncob. This is the

```
; definition tables for the objects.  Use these tables to get the
; object number of what you would like to create, and what quality
; you would like to assign that object.

                    ; number after the semicolon is
                    ; the object type number

templatetable   dw  @dataseg:objhq  ;0  Headquarters orb
    dw  @dataseg:objstet    ;1  mindbender
    dw  @dataseg:obj5da     ;2  plane  qual2=dumpy  qual3=Stealth
    dw  @dataseg:objrdr     ;3  radar
    dw  @dataseg:objtw      ;4  radar base
    dw  @dataseg:roadpiece  ;5  Runway piece  qual2= no stripes
    dw  @dataseg:tower      ;6  Control tower
    dw  @dataseg:objman ;7    Ground crew
    dw  @dataseg:objshard2  ;8
    dw  @dataseg:objshard   ;9
    dw  @dataseg:objincend  ;10
    dw  @dataseg:objdbst    ;11 KLA site
    dw  @dataseg:objvech    ;12 Vehicle  spec bit= bullet only
    dw  @dataseg:objvest    ;13 Vehicle site
    dw  @dataseg:objwallz1  ;14 Force fields
    dw  @dataseg:objwallx   ;15        "
    dw  @dataseg:objwally   ;16
    dw  @dataseg:objwallz   ;17
    dw  @dataseg:fltsm1puff ;18
    dw  @dataseg:fltsm1boom ;19
    dw  @dataseg:fltsm1bomb ;20
    dw  @dataseg:fltsm1mis  ;21
    dw  @dataseg:fltsm1aashell  ;22
    dw  @dataseg:fltsm1aa   ;23  AAA site
    dw  @dataseg:fltsm1dball    ;24  KLA
    dw  @dataseg:fltsm1     ;25
    dw  @dataseg:fltsm2     ;26
    dw  @dataseg:gms        ;27   Fuel dump
    dw  @dataseg:obj5d      ;28   Do not use!
    dw  @dataseg:objprop    ;29   Propeller
    dw  @dataseg:bullets    ;30
    dw  @dataseg:objsurv    ;31   Survey marker
    dw  @dataseg:crtr       ;32   Crater, qual 1= permanent
    dw  @dataseg:smgms      ;33   Destroyed tick
    dw  @dataseg:objhut     ;34   Tower doorway
    dw  @dataseg:objroadend ;35   runway endpieces
    dw  @dataseg:objroadend1    ;36
    dw  @dataseg:objfn1     ;37   Fences
    dw  @dataseg:objfn1a    ;38
    dw  @dataseg:objfn1b    ;39
    dw  @dataseg:objbb      ;40   Barrage Balloon
    dw  @dataseg:objbbsm    ;41   Bumble Bee
    dw  @dataseg:objmort    ;42   Mortar
    dw  @dataseg:objdhq     ;43   Blown up HQ
    dw  @dataseg:objlwallz1 ;44   Line-type forcefields
    dw  @dataseg:objlwallx  ;45
    dw  @dataseg:objlwally  ;46
    dw  @dataseg:objlwallz  ;47
    dw  @dataseg:objrv      ;48  Rescue Van
    dw  @dataseg:objshell   ;49  Shell
    dw  @dataseg:objbmr     ;50  Saucer
```

; The following is taken from the sources of Corncob. This is the (continued)

```
    dw  @dataseg:objbmrst   ;51  Saucer Site
    dw  @dataseg:objtet     ;52  Tet
    dw  @dataseg:objbmrmo   ;53  Saucer that is a mission objective
    dw  @dataseg:objslwallx ;54  Smaller line-type forcefields
    dw  @dataseg:objslwally ;55
    dw  @dataseg:objslwallz ;56
    dw  @dataseg:objal      ;57  Alien
    dw  @dataseg:objxp      ;58  Portal
    dw  @dataseg:objstep    ; 59 Small step forcefield
    dw  @dataseg:objpstep   ; 60 Small step forcefield Filled

    ; the following are the definitions of the different qualities
    ; of portals.  Ex: qual 3 takes us to base, but damages us.

xpqtbl      dw  4
quality 1   dw  11,11+10h,1,0       ; takes us back to base
quality 2   dw  3,9+10h,255,0       ; takes us to random place
quality 3   dw  12,11+10h,1,1       ; takes to base, but damages us
quality 4   dw  0,11+10h,1,-1       ; kills us instantly
quality 5   dw  11,11+10h,101h,0    ; takes us to base, fulfils mis
quality 6   dw  11,11,1,1       ; takes to base, but no flashes
quality 7   dw  7,11+10h,0,2        ; repairs plane, doesn't xpose
quality 8   dw  10,3,100h,0     ; doesn't move us, miss obj

    ; the following are the mission objectives of the different
    ; headquarter orbs.   Example: a quality 1 orb counts as a mission
        ; objective when you blow it up, a quality 4 orb will count a
mission
        ; objective when all the vehicles and vehicle sites are gone.

hqtbl   dw  2
1)  dw  0,@codeseg:killme
2)  dw  6,@codeseg:killticks
3)  dw  7,@codeseg:killdbaa
4)  dw  3,@codeseg:killvechs
5)  dw  9,@codeseg:killmetough  ; must suicide plane to destroy
6)  dw  13,@codeseg:killbmrst
7)  dw  4,@codeseg:killstets     ; kill all mindbenders
8)  dw  14,@codeseg:killal       ; kill aliens

; the following shows what qualities of shell will turn into what.
; I don't think qual=8 works

shellqtbl   dw  3       ; table of qualities for dballsites
1)      dw  13,0,6      ; vech stite
2)      dw  42,6,7      ; mortar
3)      dw  11,7,3      ; dball site
4)      dw  23,3,9      ; aa site
5)      dw  40,9,13     ; barrage balloon
6)      dw  41,13,11        ; bee swarm
7)      dw  27,11,0     ; a tick
8)      dw  23,2,2      ; 0= random
```

; The following is a table of various properties of objects. You cannot

```
; change any of these settings, it is just here for reference.
;  The numbers in the table are sums of attribute bits.  These number
are
; powers of two.  Example 512=2^9, so +512 means that this object will
; capture you if you touch it.
;
; Example: object 10  (incendiary from bomb, or mortar)  2^0=1
; 1=disappears when goes off into the distance, not saved in far tile
ram
; 2=Skip over this object when in mission builder, who'd want to create
this?
; 4=If you fly through it, object is terminated  4=bit 2  IE 2^2=4

;1  ; bit 0 means do not xfer to medium table
;2  ; bit 1 means skip over when doing mission builder creates
;4  ; bit 2 means terminate object if plane hits it
;8  ; bit 3 means it is forcefield type object
;16 ; bit 4 means it is a runway type object
;32 ; bit 5 means object is a portal
;64 ; bit 6 means object is standonable
;128    ; bit 7 means object kills you
;256    ; bit 8 means object walk height is collision + z coord
;512    ; bit 9 means object captures you

extendedtypetbl dw  0+4 ;0
    dw  0   ;1
    dw  0+4     ;2
    dw  0+4     ;3
    dw  0+4     ;4
    dw  0+4+16+64   ;5
    dw  0+4+64+256      ;6
    dw  0+4 ;7
    dw  0+2 ;8
    dw  0+2 ;9
    dw  1+2+4   ;10
    dw  0+4 ;11
    dw  4+512   ;12
    dw  0   ;13
    dw  0+4+8+64    ;14
    dw  0+4+8   ;15
    dw  0+4+8   ;16
    dw  0+4+8+64    ;17
    dw  1+2 ;18
    dw  1+2 ;19
    dw  1+2 ;20
    dw  1+2 ;21
    dw  1+2+128 ;22
    dw  0+4 ;23
    dw  128 ;24
    dw  1+2     ;25
    dw  1+2     ;26
    dw  0+4+64+256      ;27
    dw  0+2     ;28 skip over obj5d, so cursor don't dis
    dw  1   ;29
    dw  1+2 ;30
    dw  0   ;31
    dw  0+128       ;32
    dw  0       ;33
```

```
dw  0 ;34
```

```
    dw  0+16+64     ;35
    dw  0+16+64     ;36
    dw  0+4     ;37
    dw  0+4     ;38
    dw  0+4     ;39
    dw  0+128       ;40
    dw  0+128       ;41
    dw  0+4     ;42
    dw  0       ;43
    dw  0+4+8+64    ;44
    dw  0+4+8   ;45
    dw  0+4+8   ;46
    dw  0+4+8+64    ;47
    dw  1+4     ;48
    dw  0       ;49
    dw  4+64+256        ;50
    dw  0+64+256        ;51
    dw  0       ;52
    dw  4+64+256        ;53
    dw  0+4+8   ;54
    dw  0+4+8   ;55
    dw  0+4+8+64    ;56
    dw  0+512       ;57
    dw  0+4+20h     ;58
    dw  0+4+8+64    ;59
    dw  0+4+8+64    ;60
```

; Special bit definitions:, hit F9 to set an object's special bit

```
; The object's type number will turn purple to indicate that it's
special
; bit is set.
;
;  1.) Orbs with the special bit set will be gravity-attraction orbs.
;  2.) Vehicles with the special bit set will be destructible with
;        bullets or bombs ONLY, objects will last forever.  The antennas
;        of these vehicles will flash.
;  3.)  Mindbenders with this bit set will be susceptible to damage only
from
;         assassin bombs, and are invulnerable to plane-dropped bombs
;  4.)  Ground Crew with this special bit set are spies.  They wear
black and
;        their mission objectives are complete when you go near them.
;

; Complete bit (F10 toggle), is useful with the following objects
;  When the complete bit is set, the objects type number blinks
white/black.
;
;  1.) Orbs which you need, but you don't want them to be mission
objectives
;        in any way.  If you make one, but preset it's complete bit,
then
;        the player will not need to accomplish that orb's objective to
;        finish a theater.  This is useful when you want an orb to power
;       the local force fields, but you don't want the pilot to make the
;       mission to be trivial because he can just blow the orb to drop
the
;       force fields.  The solution is to make a kill-me-tough orb with
it's
;       complete bit already set.  Then the pilot can't kill it without
;       sacrificing a plane, but the player doesn't NEED to kill that
orb
;       in order to complete the theater.
;
;  2.)  Portals which may be desired, but not count as mission
objectives.
;       Just preset their complete bits in order to make portals which
the
;       the player may use, but do not have to be flown through to
satisfy
;       an MO.
;
;  3.)  Mission objective-type saucers where you want one to stay put
like
;         a mission objective saucer, but you don't want it to count as
;         an MO.
```